BIZEV-04: Revenue Impact Analysis

> **Series:** BIZEV — Business Events & Funnel Analysis | **Notebook:** 4 of 7 | **Created:** March 2026 | **Last Updated:** 11/11/2026

Overview

When a Dynatrace detected problem fires, the immediate question from stakeholders is: "What is the business impact?" This notebook demonstrates how to correlate Dynatrace Intelligence problems with business event data to quantify revenue loss during incidents, measure SLA impact, compare incident periods against baselines, and filter analysis to business hours. The goal is to translate technical incidents into business language that executives and product owners understand.

Table of Contents

1. Connecting Problems to Business Impact
2. Revenue During Incidents vs Baseline
3. Calculating Revenue Loss
4. Business Hours Filtering
5. Day-Over-Day Business Comparison
6. SLA Impact Measurement
7. Building an Impact Timeline
8. Summary and Next Steps

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
Permissions	`storage:bizevents:read`, `storage:events:read`
Data	Business events with revenue/amount fields; detected problems data
Knowledge	BIZEV-01 through BIZEV-03, detected problems concepts

1. Connecting Problems to Business Impact

Dynatrace Intelligence detects infrastructure and application problems. Business events capture transactions. By correlating the two, you can answer: "During the 2-hour outage on Tuesday, how many transactions were lost?"

Approach

1. Fetch detected problems with their time windows (`event.start` to `event.end`)
2. Fetch business events during the same time windows
3. Compare business event volume during the problem vs a baseline period
4. Calculate the delta as "lost" transactions or revenue

Let's start by examining recent closed problems.

```
// Recent closed detected problems with duration
fetch dt.davis.problems, from:-7d
| filter event.status == "CLOSED"
| fieldsAdd duration_min = resolved_problem_duration / 1m
| fieldsKeep display_id, event.name, event.start, event.end, duration_min,
event.category
| sort event.start desc
| limit 10
```

```
// Business event volume per hour over the past 7 days
// Overlay with the concurrent-open-problems query in Section 7 to line up
dips with incidents
fetch bizevents, from:-7d
| makeTimeseries biz_events = count(), interval:1h
```

2. Revenue During Incidents vs Baseline

To measure impact, compare the business event rate during an incident against a "normal" baseline. The baseline can be:

- The same time period on the previous day
- The average for that hour of the week
- The 7-day rolling average

```
// Compare today's business event volume vs yesterday (two 24-hour totals)
fetch bizevents, from:-24h
| summarize today_count = count()
| append [
    fetch bizevents, from:-48h, to:-24h
    | summarize yesterday_count = count()
  ]
```

```
// Hourly business event comparison — today vs same day last week
fetch bizevents, from:-24h
| fieldsAdd hour = formatTimestamp(timestamp, format: "HH")
| summarize today_events = count(), by:{hour}
| append [
    fetch bizevents, from:-192h, to:-168h
    | fieldsAdd hour = formatTimestamp(timestamp, format: "HH")
    | summarize last_week_events = count(), by:{hour}
  ]
| sort hour asc
```

3. Calculating Revenue Loss

If your business events include an `amount` or `total` field, you can calculate actual revenue impact.

> **Note:** Replace `amount` with the actual field name in your business events that carries the transaction value.

```
// Total revenue from completed orders in the last 24 hours
fetch bizevents, from:-24h
| filter event.type == "com.myapp.order.completed"
| filter isNotNull(amount)
| summarize {total_revenue = sum(toDouble(amount)),
           order_count = count(),
           avg_order_value = avg(toDouble(amount))}
```

```
// Revenue per hour — detect dips that correlate with incidents
fetch bizevents, from:-24h
| filter event.type == "com.myapp.order.completed"
| filter isNotNull(amount)
| makeTimeseries hourly_revenue = sum(toDouble(amount)), interval:1h
```

```
// Hourly revenue — pick the incident hours and compare them with the
surrounding hours
// (or with the same hours from the day-over-day query in Section 5) to
estimate the loss
fetch bizevents, from:-24h
| filter event.type == "com.myapp.order.completed"
| filter isNotNull(amount)
| fieldsAdd hour = getHour(timestamp)   // UTC unless you pass timezone: — see
Section 4
| summarize {hourly_revenue = sum(toDouble(amount)),
           order_count = count()}, by:{hour}
| sort hour asc
```

4. Business Hours Filtering

Not all hours are equal. An incident at 3 AM on a Sunday has different business impact than one at 10 AM on a Tuesday. Filter analysis to business hours for more meaningful impact assessment.

> **Pass the timezone explicitly.** `getHour()` and `getDayOfWeek()` evaluate in the query's timezone unless you give them one — UTC when the query runs through the API or a workflow — so "9 to 5" silently shifts by your UTC offset. Give both a

> `timezone:` parameter (the queries below use `America/New_York`; substitute your own). `getDayOfWeek()` numbers days ISO-style: 1 = Monday … 7 = Sunday.

```
// Business events during business hours only (Mon-Fri, 9 AM - 5 PM)
fetch bizevents, from:-7d
| fieldsAdd dow = getDayOfWeek(timestamp, timezone: "America/New_York"),
           hour = getHour(timestamp, timezone: "America/New_York")
| filter dow &gt;= 1 and dow &lt;= 5 and hour &gt;= 9 and hour &lt; 17
| summarize business_hours_events = count(),
           by:{day = bin(timestamp, 24h)}
| sort day asc
```

```
// Compare business hours revenue vs after-hours revenue
fetch bizevents, from:-7d
| filter event.type == "com.myapp.order.completed"
| filter isNotNull(amount)
| fieldsAdd dow = getDayOfWeek(timestamp, timezone: "America/New_York"),
           hour = getHour(timestamp, timezone: "America/New_York")
| fieldsAdd period = if(dow &gt;= 1 and dow &lt;= 5 and hour &gt;= 9 and hour
&lt; 17,
                        then: "Business Hours",
                        else: "After Hours")
| summarize {total_revenue = sum(toDouble(amount)),
           order_count = count()}, by:{period}
```

5. Day-Over-Day Business Comparison

Compare today's business metrics against yesterday to quickly spot anomalies.

```
// Day-over-day comparison: today vs yesterday order counts
fetch bizevents, from:bin(now(), 24h)
| filter event.type == "com.myapp.order.completed"
| summarize today_orders = count()
| append [
    fetch bizevents, from:bin(now(), 24h) - 24h, to:bin(now(), 24h)
    | filter event.type == "com.myapp.order.completed"
    | summarize yesterday_orders = count()
  ]
```

```
// Week-over-week trend — daily order counts for the past 2 weeks
fetch bizevents, from:-14d
| filter event.type == "com.myapp.order.completed"
| fieldsAdd day = bin(timestamp, 24h)
| summarize daily_orders = count(), by:{day}
| sort day asc
```

6. SLA Impact Measurement

Service Level Agreements (SLAs) often include business availability targets. By correlating problem duration with business event volume, you can measure whether SLAs were met.

> **Impact-hours are not downtime, and cannot become an SLA percentage.** Problems overlap — Dynatrace opens one per affected entity and many run concurrently — so summing their durations counts the same wall-clock minutes many times over. On a live tenant over 7 days (08/25/2026) the sum reached **3,748 hours** against a 40-hour business week; the SLA formula that previously lived here returned **-9,271% availability**. The sum is genuinely useful as *entity-hours of impact* — it is what the next query reports — but an availability or SLA figure has to come from an **SLO**, which measures a good-events ÷ total-events ratio rather than subtracting problem time. See **SLO-02** for defining the SLI and **SLO-03** for error budgets.

```
// Total problem duration this week (hours of impact)
fetch dt.davis.problems, from:-7d
| filter event.status == "CLOSED"
| filter dt.davis.is_duplicate == false
// Exclude problems raised during maintenance windows (planned work); drop
this line
// if your SLA counts maintenance time
| filter maintenance.is_under_maintenance == false
| fieldsAdd duration_hours = resolved_problem_duration / 1h
| summarize {total_problem_hours = sum(duration_hours),
           problem_count = count(),
           avg_mttr_hours = avg(duration_hours)}
```

```
// Problem impact in business terms. NOTE: impact_hours counts overlapping
problems
// separately, so it is entity-hours of impact, NOT wall-clock downtime — it
cannot be
// subtracted from a business-hours window to yield an SLA percentage.
fetch dt.davis.problems, from:-7d
| filter event.status == "CLOSED"
| filter dt.davis.is_duplicate == false
// Exclude problems raised during maintenance windows (planned work); drop
this line
// if your SLA counts maintenance time
| filter maintenance.is_under_maintenance == false
| fieldsAdd duration_hours = resolved_problem_duration / 1h
// affected_entity_ids is deprecated in favor of smartscape.affected_entities.
Neither covers every
// problem: the classic field names only ENVIRONMENT-0000000000000001 for
entities that exist only in
// Smartscape (AWS EC2 instances, for example), and the Smartscape field is
empty on many process
// group instance problems. Take Smartscape node IDs where a problem has them,
classic IDs where not.
| fieldsAdd ss_ids = toString(smartscape.affected_entities[][id]),
            classic_ids = if(isNull(smartscape.affected_entities),
affected_entity_ids)
// The ID fields are arrays: countDistinctExact on them counts distinct
ARRAYS, not
// entities — flatten the collected arrays first, then count distinct entity
IDs.
| summarize {impact_hours = sum(duration_hours),
           problem_count = count(),
           ss_arrays = collectArray(ss_ids),
           classic_arrays = collectArray(classic_ids),
           avg_problem_hours = avg(duration_hours)}
| fieldsAdd affected_entities =
arraySize(arrayRemoveNulls(arrayDistinct(arrayConcat(arrayFlatten(ss_arrays),
arrayFlatten(classic_arrays)))))
| fieldsRemove ss_arrays, classic_arrays
```

7. Building an Impact Timeline

An impact timeline overlays business event volume with detected problems to visualize how incidents affect business operations.

```
// Business event volume over the past 7 days
// Overlay with problem occurrences to see impact
fetch bizevents, from:-7d
| filter event.type == "com.myapp.order.completed"
| makeTimeseries orders = count(), interval:1h
```

```
// Concurrent open problems over time — overlay with order volume chart
fetch dt.davis.problems, from:-7d
| makeTimeseries open_problems = count(),
    spread: timeframe(from: event.start, to: coalesce(event.end, now()))
```

8. Summary and Next Steps

In this notebook, you learned:

- **Problem-to-business correlation** — Connecting detected problems to business event data
- **Revenue during incidents** — Comparing incident periods against baselines
- **Revenue loss estimation** — Using transaction amounts to quantify financial impact
- **Business hours filtering** — Scoping analysis to when it matters most
- **Day-over-day comparison** — Detecting business anomalies via historical comparison
- **Impact hours** — Entity-hours of problem impact, and why they are not an SLA percentage
- **Impact timelines** — Overlaying problems with business metrics

Next Steps

- **BIZEV-05: KPIs and Metrics** — Define ongoing business health indicators
- **BIZEV-06: Executive Reporting** — Build executive-ready reports combining these techniques

References

- Davis Problems app (DT docs)
- Dynatrace Business Analytics

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.